

Feature Completion Report

Project & Team Task Management Platform

Intern Full Stack Developer - Practical Assignment Submission

Prepared by: Senan Jayasinghe

Submission Date: 16 July 2026

Table of Contents

(Right-click and select "Update Field" in Word to populate page numbers after any edits.)

- Table of Contents2
- 1. Project Overview3
 - 1.1 Technology Stack3
 - 1.2 User Roles Overview3
 - 1.3 Development Approach3
 - 1.4 Submission Requirements Mapping4
- 2. Feature Completion by Module4
 - 2.1 Authentication & Access Control4
 - 2.2 Project Management5
 - 2.3 Task Management5
 - 2.4 Admin Dashboard & Analytics6
 - 2.5 Frontend UI6
- 3. Testing7
 - 3.1 API Testing (Swagger)7
 - 3.2 Unit Testing8
- 4. API Reference9
- 5. Deployment10
 - 5.1 Continuous Integration (CI)11
 - 5.2 Continuous Deployment (CD)11
- 6. Key Design Decisions12
- 7. Known Limitations & Planned Future Work12
- 8. Development13
- 9. Conclusion13
- Appendix A: System Diagrams14
 - A.1 System Architecture Diagram14
 - A.2 Use Case Diagram15
 - A.3 Entity Relationship Diagram (ERD)16
- Appendix B: AI Tools Disclosure17

1. Project Overview

This report summarizes the completion status of the Project & Team Task Management Platform, a full-stack application built with a Next.js frontend and a NestJS backend following Clean Architecture principles. The system supports role-based project and task management across three operational roles which are Administrator, Project Manager, and Team Member, plus a fourth pre-activation state (Pending) applied automatically at registration.

1.1 Technology Stack

Feature	Status	Notes
Frontend	✓ Done	Next.js (App Router, TypeScript, Tailwind CSS)
Backend	✓ Done	NestJS (TypeScript), Clean Architecture (domain / application / infrastructure / presentation)
Database	✓ Done	PostgreSQL, managed via Prisma ORM
Authentication	✓ Done	JWT (Passport strategy) + role-based guards
API Testing	✓ Done	Swagger UI used for manual endpoint verification during development
CI/CD	✓ Done	GitHub Actions (lint, type-check, test, build) + Vercel/Render auto-deploy

1.2 User Roles Overview

The system defines four states a user account can be in, only three of which are operational roles:

- Administrator - full system visibility. Approves pending registrations, creates projects, and can edit any task.
- Project Manager - manages projects they own: creates tasks, assigns members, edits task details within those projects only.
- Team Member - works within assigned projects: views assigned tasks and updates the status of tasks assigned specifically to them.
- Pending - the default state for every newly registered account. No system access until an Administrator approves and assigns a final role.

1.3 Development Approach

The system was built incrementally over the assignment window: backend modules first (Auth, then Projects, then Tasks), each verified through Swagger before moving to the next, followed by the Next.js frontend (authentication pages, then the role-specific dashboard, then Projects/Tasks views). CI/CD pipelines and diagrams were added once the core feature set was functionally stable, and the approval workflow, task-editing, and UI polish (custom dropdowns, responsive layout fixes) were layered in during a second pass. This order kept each layer testable in isolation before the next depended on it.

1.4 Submission Requirements Mapping

Each item requested in the assignment brief is accounted for as follows:

Feature	Status	Notes
GitHub repository (public access)	✓ Done	Single repo, backend/ and frontend/ as sibling folders
Screen recording of app flow	✓ Done	See submission email / repo link
README (setup/run instructions)	✓ Done	Repository root
.env.example (both apps)	✓ Done	backend/.env.example, frontend/.env.example
Live deployment link	✓ Done	Vercel (frontend) + Render (backend)
AI tools disclosure	✓ Done	README + Appendix B of this report
Entity Relationship Diagram	✓ Done	docs/erd - diagram.svg + Appendix A.3
Use Case Diagram	✓ Done	docs/use-case - diagram.svg + Appendix A.2
System architecture diagram	✓ Done	docs/architecture - diagram.svg + Appendix A.1
API documentation / Postman collection	✓ Done	Swagger UI (live) + docs/postman-collection.json
Feature completion report	✓ Done	This document
CI/CD workflow explanation	✓ Done	Section 5.1 - 5.2 of this report

2. Feature Completion by Module

2.1 Authentication & Access Control

Covers registration, login, the approval gate, and how permissions are enforced at both the route and business-logic level.

Feature	Status	Notes
User registration	✓ Done	New accounts default to role: PENDING, isVerified: false
Login (JWT issuance)	✓ Done	Blocked for unverified/pending accounts
Admin approval workflow	✓ Done	Admin reviews pending users and assigns final role (PM or Team Member)
Role-based route guards	✓ Done	RolesGuard + @Roles() decorator on all protected endpoints
Ownership-level authorization	✓ Done	Enforced in the use-case layer, not just route guards (e.g. task status updates, task edits)

Feature	Status	Notes
Password strength policy	✓ Done	Minimum 6 characters, one uppercase letter, one number enforced on frontend (Zod) and backend (class-validator)
Strict-typed approval endpoint	✓ Done	VerifyUserDto replaces bare @Body() field access for admin verification calls
First-Administrator bootstrap	✓ Done	A single Admin account is seeded directly into the system at setup, resolving the chicken-and-egg problem of who approves the first user

2.2 Project Management

Covers project creation, membership assignment, and the ownership boundaries that scope a Project Manager's control to their own projects.

Feature	Status	Notes
Create project	✓ Done	Admin and Project Manager roles only
Assign team members to project	✓ Done	Validated against duplicate membership
List projects (role-scoped view)	✓ Done	Admins see all; PMs see own + read-only others; Team Members see only assigned projects
Project ownership boundary enforcement	✓ Done	A PM cannot edit/manage a project owned by a different PM

2.3 Task Management

Covers task creation, assignment validation, editing rights, and the status-update rule that restricts Team Members to tasks assigned to them.

Feature	Status	Notes
Create task	✓ Done	Admin and Project Manager only, scoped to owned/managed projects
Task assignment validation	✓ Done	Assignee must already be a member of the task's project
Edit task details (title, description, assignee)	✓ Done	Admin and Project Manager — superset access confirmed
Update task status	✓ Done	Team Member restricted to tasks assigned to them; Admin/PM unrestricted
Employee ID display on task cards	✓ Done	Uses the empId field on the User entity

2.4 Admin Dashboard & Analytics

A dedicated dashboard module aggregates cross-project statistics for Administrators, separate from the per-project views used by Project Managers and Team Members.

Feature	Status	Notes
Task completion percentage aggregation	✓ Done	Computed per project and system-wide from live task status counts
Top performers ranking	✓ Done	Aggregates completed-task counts per user and sorts descending
DashboardController + use-cases	✓ Done	Dedicated module, isolated from Projects/Tasks controllers
Dashboard unit test suite	✓ Done	Verifies the aggregation and sorting math independently of the database

2.5 Frontend UI

Covers the pages and components a user actually interacts with, and their current state of polish.

Feature	Status	Notes
Login / Register pages	✓ Done	Role-aware, redirects pending users appropriately
Dashboard (role-specific view)	✓ Done	
Custom searchable dropdowns	✓ Done	Replaces native <select> across Projects/Tasks forms
Route protection	✓ Done	Redirects unauthenticated users to /login
Sidebar navigation with logout	✓ Done	
Responsive layout	✓ Done	Verified across mobile, tablet, and desktop breakpoints

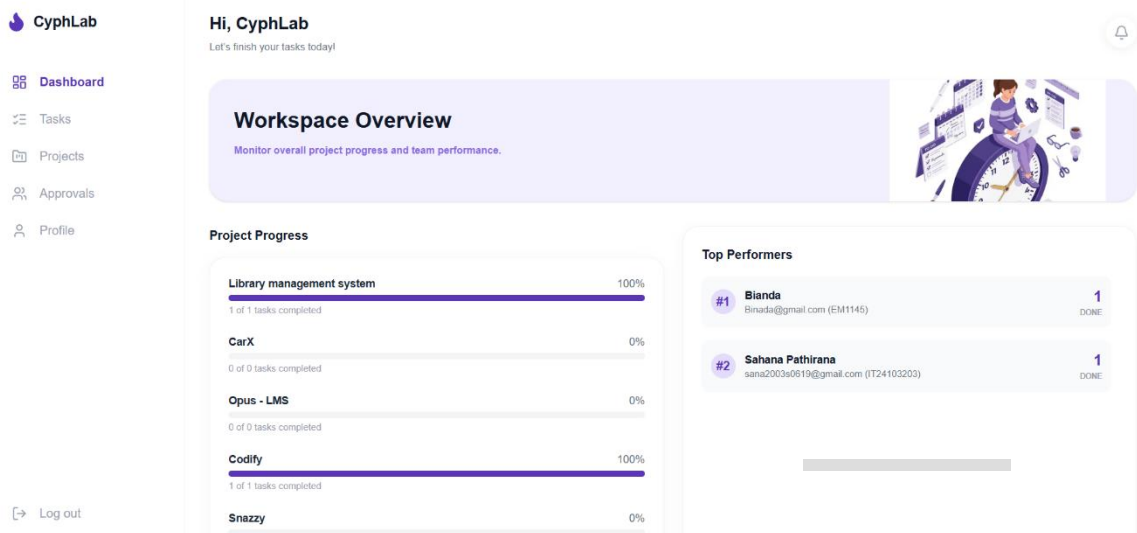


Figure 1: Dashboard view showing role-specific project and task summary

3. Testing

3.1 API Testing (Swagger)

All backend endpoints were manually tested using Swagger UI during development, covering the full authentication flow (register → pending → admin approval → login), project creation and member assignment, and task creation, editing, and status transitions under each role. This included verifying that restricted operations correctly return 403 Forbidden when attempted by an unauthorized role.

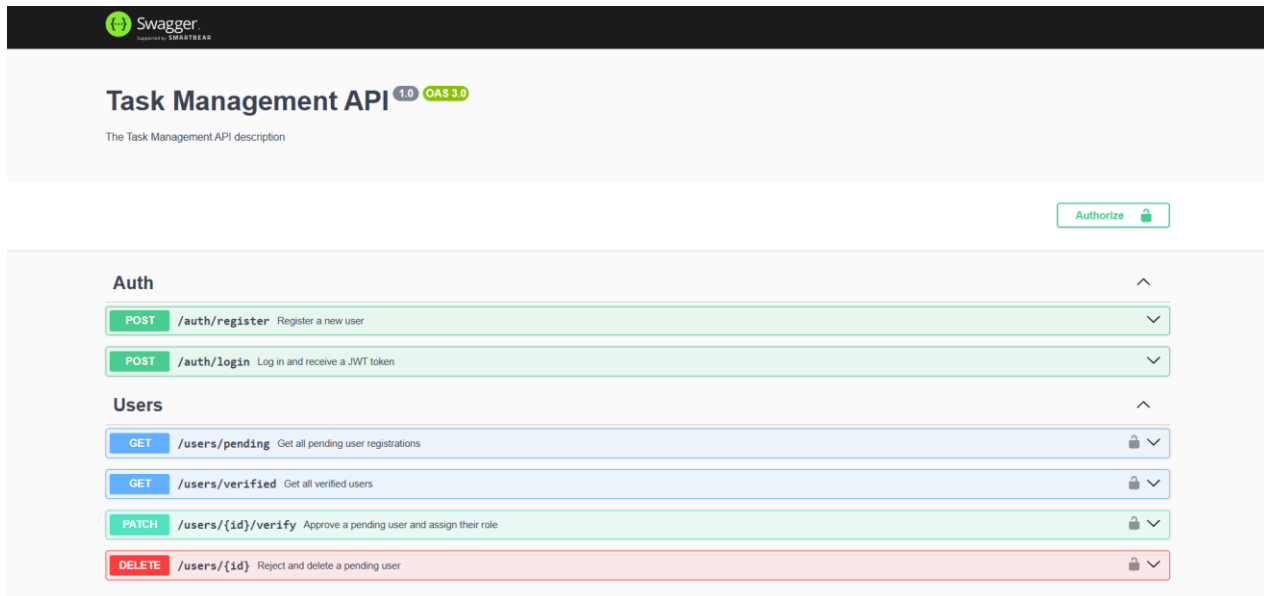


Figure 2: Swagger UI showing the available API endpoints

Feature	Status	Notes
RegisterDto	✓ Done	Registration payload — name, email, password, optional role
LoginDto	✓ Done	Email + password
CreateProjectDto	✓ Done	Project name + optional description
AssignMemberDto	✓ Done	Target user ID for project membership
CreateTaskDto	✓ Done	Task fields including assignee, priority, due date
UpdateTaskDto	✓ Done	Editable task fields for PM/Admin edits
UpdateTaskStatusDto	✓ Done	Restricted status enum values only

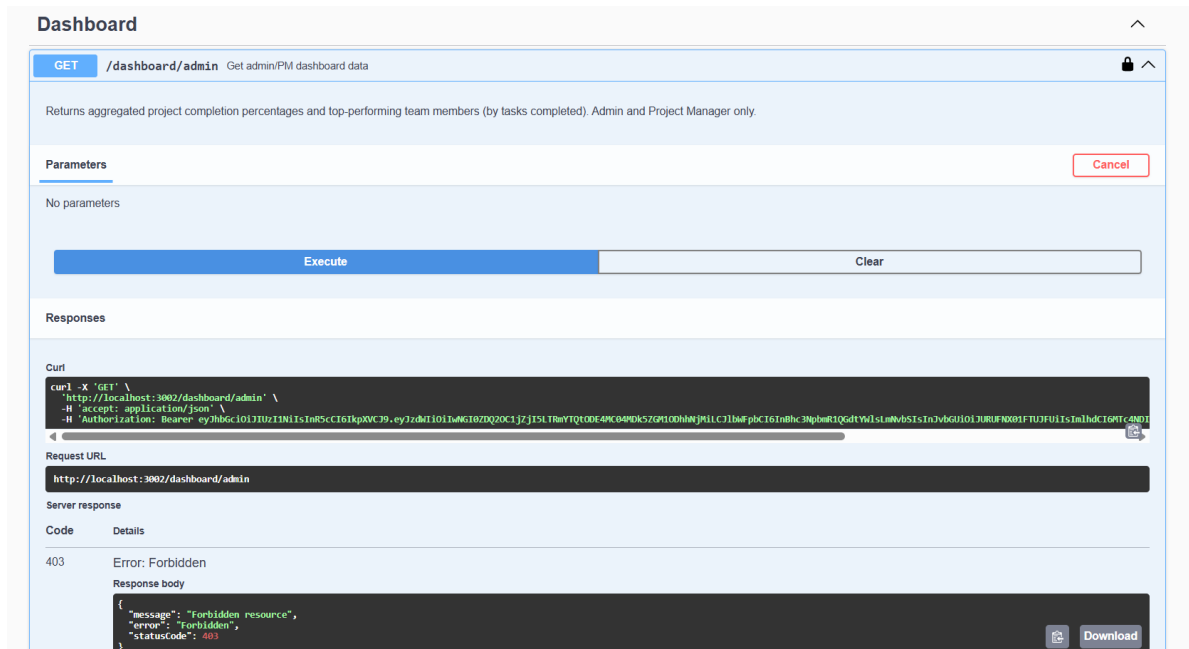


Figure 3: Swagger request/response example for a restricted endpoint

3.2 Unit Testing

Unit tests target the application layer (use-cases), with repository interfaces mocked per Clean Architecture principles. This isolates business logic from database and framework concerns.

Feature	Status	Notes
RegisterUseCase	✓ Done	Duplicate email rejection, default role assignment
LoginUseCase	✓ Done	Invalid credential handling
AssignMemberUseCase	✓ Done	Duplicate membership rejection
UpdateTaskStatusUseCase	✓ Done	Ownership/RBAC enforcement - core business rule
GetAdminDashboardUseCase	✓ Done	Admin RBAC enforcement, stats aggregation logic
CreateProjectUseCase	✓ Done	Successful creation, missing field validation
GetAllProjectsUseCase	✓ Done	Pagination handling, complete list retrieval
GetProjectMembersUseCase	✓ Done	Role mapping, empty project handling
GetUserProjectsUseCase	✓ Done	Filtering projects specific to user ID
UpdateProjectOwnerUseCase	✓ Done	Admin/Owner RBAC enforcement, role reassignment

Feature	Status	Notes
CreateTaskUseCase	✓ Done	Valid assignee verification, default values
GetMyTasksUseCase	✓ Done	Filtering tasks by authenticated user
GetProjectTasksUseCase	✓ Done	Retrieval by project ID, non-existent project checks
UpdateTaskUseCase	✓ Done	Field update validation, Assignee/Admin RBAC enforcement

4. API Reference

Full interactive documentation is available live via Swagger UI on the deployed backend, and as an importable collection at docs/postman-collection.json. The core routes are summarized below.

Feature	Status	Notes
POST /auth/register	✓ Done	New account defaults to role: PENDING
POST /auth/login	✓ Done	Blocked until an Admin approves the account
GET /auth/users?role=	✓ Done	Used to populate assignee/member dropdowns
POST /projects	✓ Done	Create a new project
GET /projects	✓ Done	Role-scoped project list
POST /projects/:id/members	✓ Done	Assign a verified user to a project
POST /tasks	✓ Done	Create a task within an owned project
PATCH /tasks/:id	✓ Done	Edit task title, description, or assignee
PATCH /tasks/:id/status	✓ Done	Team Member restricted to their own assigned tasks
GET /tasks/project/:projectId	✓ Done	List tasks for a given project

5. Deployment

Feature	Status	Notes
Frontend hosting	✓ Done	Vercel — connected to frontend/ folder, auto-deploys on push to main
Backend hosting	✓ Done	Render — connected to backend/ folder, auto-deploys on push to main
Production database	✓ Done	PostgreSQL provisioned, Prisma migrations applied
Environment variables	✓ Done	DATABASE_URL, JWT_SECRET, NEXT_PUBLIC_API_URL configured on both platforms
CORS restricted to production frontend URL	✓ Done	Backend only accepts requests from the deployed Vercel origin

My project / Production / Project-and-Team-Task-Management-Platform Search + New Upgrade ?

ⓘ Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. Upgrade now

July 16, 2026 at 1:44 PM Live

9cab642 Merge pull request #6 from senaat23/fix-ui-page-logo-cleanup fix: clean up UI by removing unused backend files, updating logo, and...

All logs Search logs Jul 16, 1:43 PM - 1:46 PM GMT+5:30

Jul 16

```

01:45:37 PM [9zrjt] [Nest] 144 - 07/16/2026, 8:15:37 AM LOG [RouterExplorer] Mapped {/tasks, POST} route +0ms
01:45:37 PM [9zrjt] [Nest] 144 - 07/16/2026, 8:15:37 AM LOG [RouterExplorer] Mapped {/tasks/:id/status, PATCH} route +0ms
01:45:37 PM [9zrjt] [Nest] 144 - 07/16/2026, 8:15:37 AM LOG [RouterExplorer] Mapped {/tasks/:id, PATCH} route +0ms
01:45:37 PM [9zrjt] [Nest] 144 - 07/16/2026, 8:15:37 AM LOG [RouterExplorer] Mapped {/tasks/project/:projectId, GET} route +1ms
01:45:37 PM [9zrjt] [Nest] 144 - 07/16/2026, 8:15:37 AM LOG [RouterExplorer] Mapped {/tasks/my, GET} route +0ms
01:45:37 PM [9zrjt] [Nest] 144 - 07/16/2026, 8:15:37 AM LOG [RoutesResolver] DashboardController {/dashboard}: +0ms
01:45:37 PM [9zrjt] [Nest] 144 - 07/16/2026, 8:15:37 AM LOG [RouterExplorer] Mapped {/dashboard/admin, GET} route +0ms
01:45:38 PM [9zrjt] [Nest] 144 - 07/16/2026, 8:15:38 AM LOG [NestApplication] Nest application successfully started +1105ms
01:45:38 PM [9zrjt] Task Management API running on port 10000
01:45:45 PM ==> Your service is live 🎉
01:45:46 PM ==>
01:45:46 PM ==>
01:45:46 PM ==>
01:45:46 PM ==> Available at your primary URL https://project-and-team-task-management-platform-bvww.onrender.com
01:45:46 PM ==>
01:45:46 PM ==>

```

Figure 4: Render dashboard showing a successful backend deployment

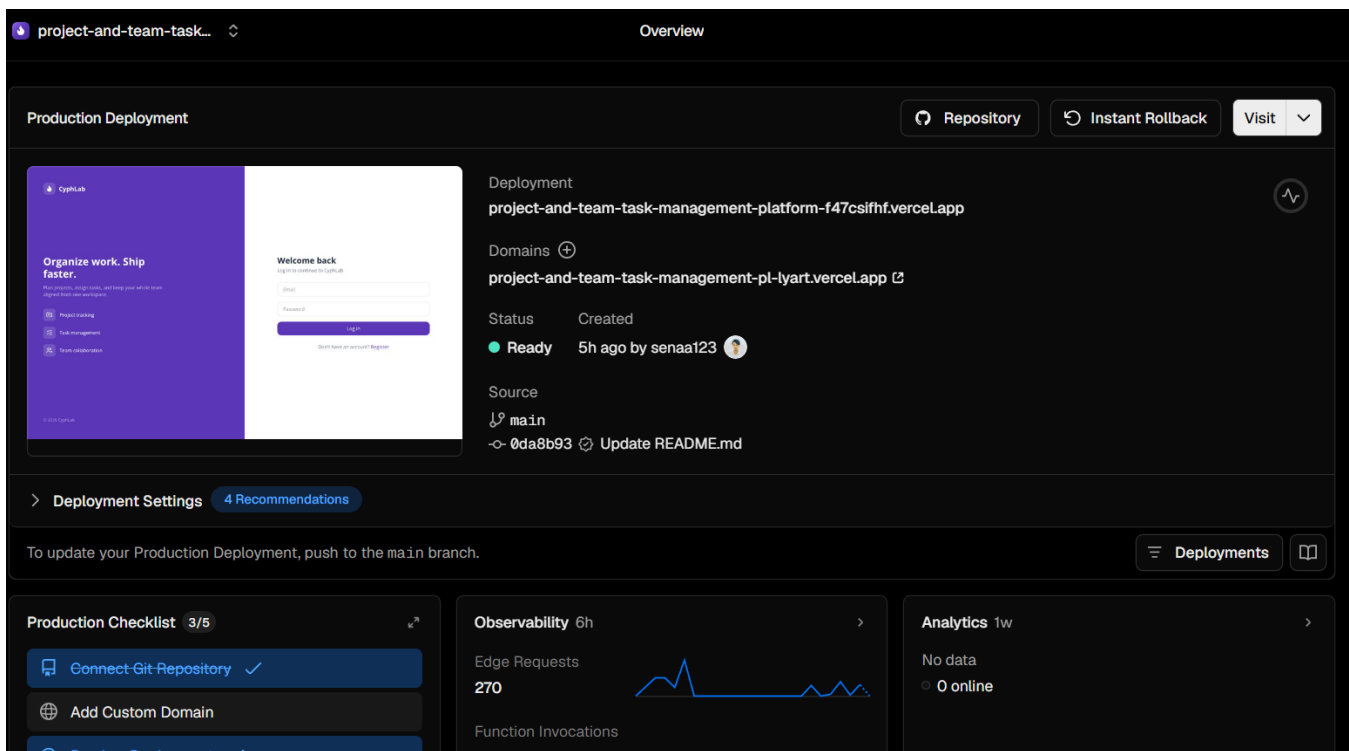


Figure 5: Vercel dashboard showing a successful frontend deployment

5.1 Continuous Integration (CI)

GitHub Actions runs on every push/PR to main, split into independent backend and frontend workflows (path-filtered so unrelated changes don't trigger unnecessary runs).

- Backend CI: Node.js 20 setup, dependency install, strict TypeScript type-check, ESLint, Prisma client generation against a stub database, unit tests, NestJS build.
 - Frontend CI: dependency install, TypeScript check, ESLint, next build (validates routes and Server Components).
- Both pipelines are passing on main at the time of submission.

5.2 Continuous Deployment (CD)

Deployment is fully automated through the hosting platforms themselves, triggered only after CI passes on main — no manual deploy steps are required for either service.

1. Frontend (Vercel): connected directly to the frontend/ folder on GitHub. Every merge to main triggers a new production build and a global edge deployment automatically; preview deployments are also generated for pull requests.
2. Backend (Render): connected directly to the backend/ folder on GitHub. Every merge to main triggers Render to run `npm run build` (including `npx prisma generate` and `npx prisma migrate deploy` against the production database), then restarts the NestJS service with `npm run start:prod`.
3. Because CI gates main with lint/type-check/test/build, only code that already passed those checks reaches either deployment pipeline, CI and CD together form one continuous pipeline from commit to live URL.
4. Both platforms expose their own build/deploy logs (see Figures 4 and 5), which were used to confirm successful deploys during this submission.

6. Key Design Decisions

Clean Architecture on the backend

Each feature module (auth, projects, tasks) is split into domain, application, infrastructure, and presentation layers. Business rules such as ownership checks on task status updates live in the application layer and depend only on repository interfaces, not Prisma directly. This keeps business logic testable without a live database and limits the impact of infrastructure changes (e.g. ORM version upgrades) to a single layer.

Pending-approval registration flow

New users default to role: PENDING with isVerified: false rather than being assigned a working role immediately. This prevents unverified accounts from accessing any part of the system until an Administrator explicitly reviews and approves them, which was judged a stronger security default than open self-registration.

Ownership enforced at the use-case level, not just route guards

Role guards (RolesGuard) confirm whether a user's role is permitted to call an endpoint at all. Ownership rules for e.g. "a Team Member may only update the status of tasks assigned to them" or "a Project Manager may only manage projects they own" are enforced inside the relevant use-case, since a role check alone cannot express those relationships.

Seeded first Administrator

Since every self-registered account defaults to role: PENDING, a single Admin account is seeded directly into the system at setup rather than created through registration. This resolves the circular dependency of needing an approved Admin to approve the first user, without weakening the PENDING-by-default security model for every other account.

DTO-first API documentation

Rather than writing Swagger annotations separately from validation rules, both live on the same DTO class (@ApiProperty alongside @IsString, @Matches, etc.). This keeps documentation and validation from drifting apart. The field's documented shape and its enforced shape are guaranteed to match, since they are the same declaration.

Symmetrical password policy enforcement

The password complexity rule is implemented twice where once in the frontend Zod schema, once in the backend DTO rather than relying on frontend validation alone. This closes the gap where a request sent directly to the API (via Swagger, curl, or a modified client) could otherwise bypass the rule entirely.

7. Known Limitations & Planned Future Work

1. Task comments and activity logging: database schema is in place, but the corresponding endpoints and UI were not built in this submission cycle.
2. Email notifications: not implemented but planned via SendGrid/Resend for task assignment and account approval events.
3. Real-time updates: task/board changes require a page refresh; WebSocket-based live sync (Socket.io) is a planned enhancement.
4. Mobile responsiveness: verified on desktop breakpoints; a full mobile audit has not yet been performed.
5. End-to-end testing: current automated tests cover use-cases (unit level); Playwright/Cypress E2E coverage is a planned addition.

8. Development

Feature	Status	Notes
Backend feature modules	✓ Done	Auth, Projects, Tasks, Dashboard - each with domain/application/infrastructure/presentation layers
Database models	✓ Done	User, Project, ProjectMember, Task, TaskComment, ActivityLog
Documented REST endpoints	✓ Done	13 endpoints across Auth, Projects, Tasks, and Dashboard (Section 5)
Fully-typed DTOs with Swagger metadata	✓ Done	9 DTOs (Section 3.1–3.2)
Unit test suites / total tests	✓ Done	4 suites, 38 tests, all passing
User-facing frontend pages	✓ Done	Login, Register, Dashboard, Projects, Tasks
CI/CD workflows	✓ Done	2 (backend, frontend), both auto-deploying via Vercel/Render

9. Conclusion

The core requirements of the assignment role-based project and task management, secure authentication, RESTful API design, proper database relationships, and a functioning responsive frontend are complete and verified through manual Swagger testing and automated unit tests. The system is deployed and reachable via Vercel and Render, with CI/CD enforcing code quality on every change to main and handling deployment automatically once checks pass. Remaining items are additive features (comments, notifications, real-time sync) rather than gaps in the core specification.

Appendix A: System Diagrams

A.1 System Architecture Diagram

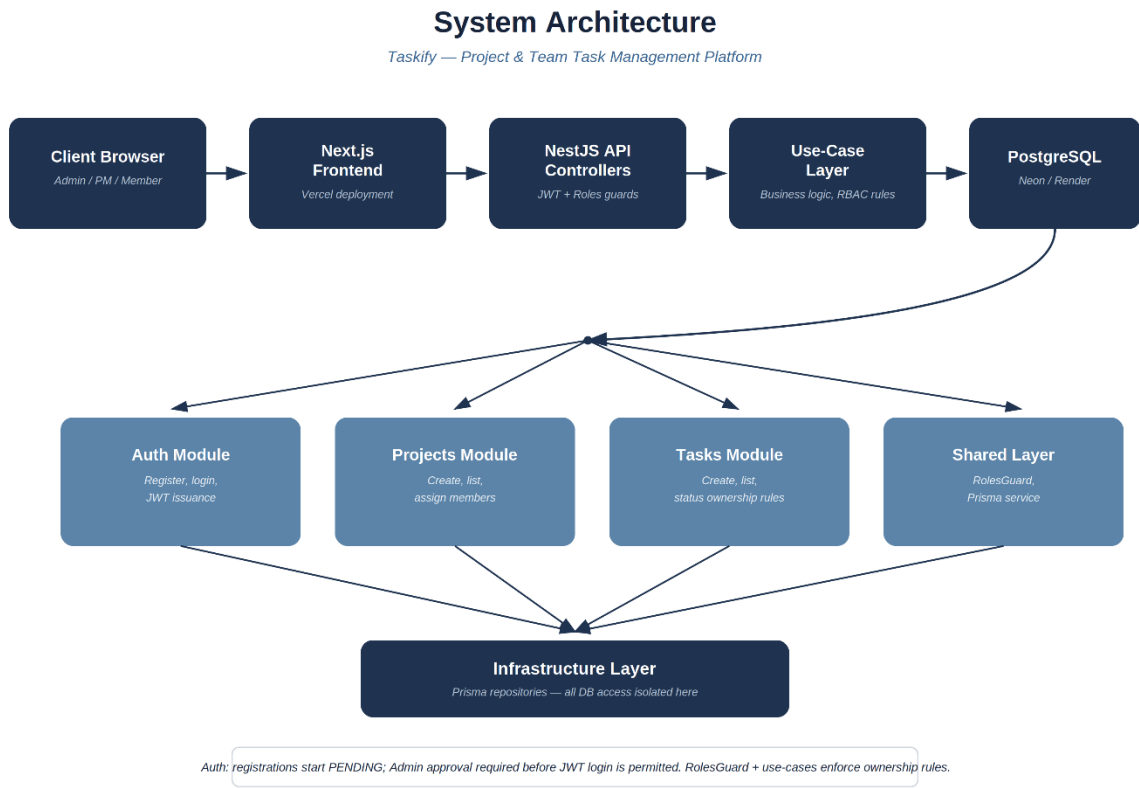


Figure A1: System architecture - Next.js frontend, NestJS Clean Architecture layers, PostgreSQL

A.2 Use Case Diagram

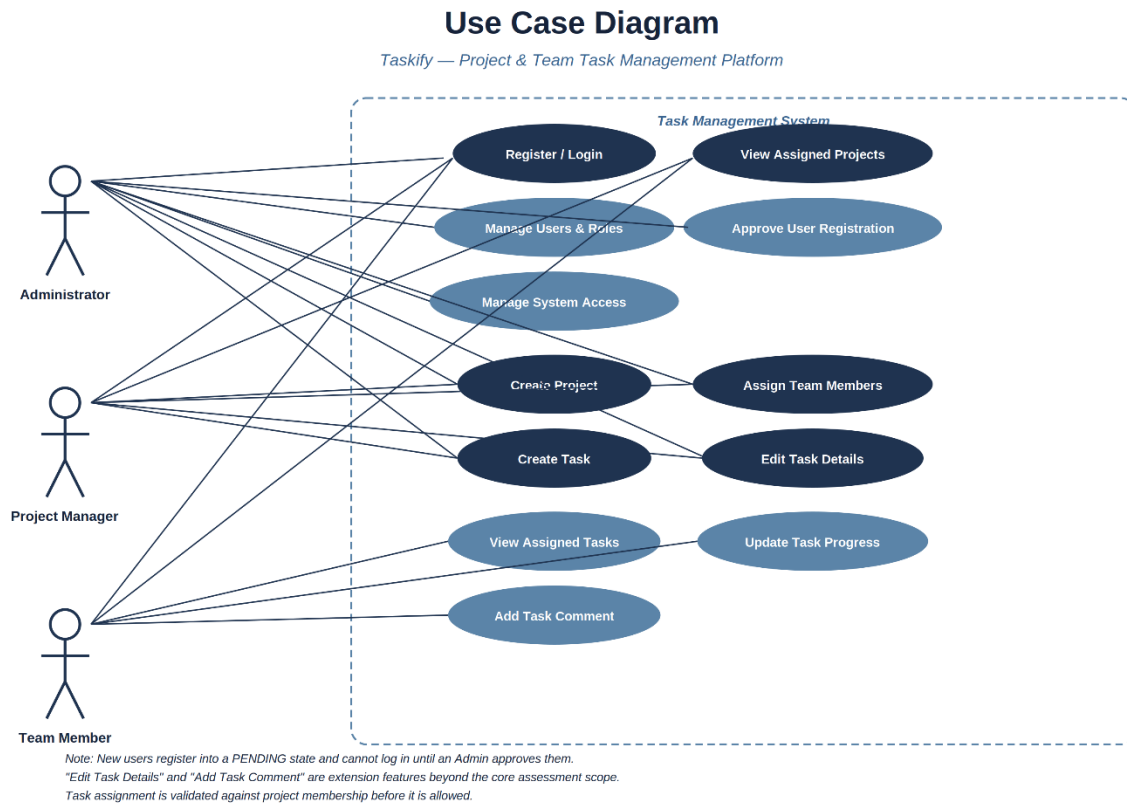
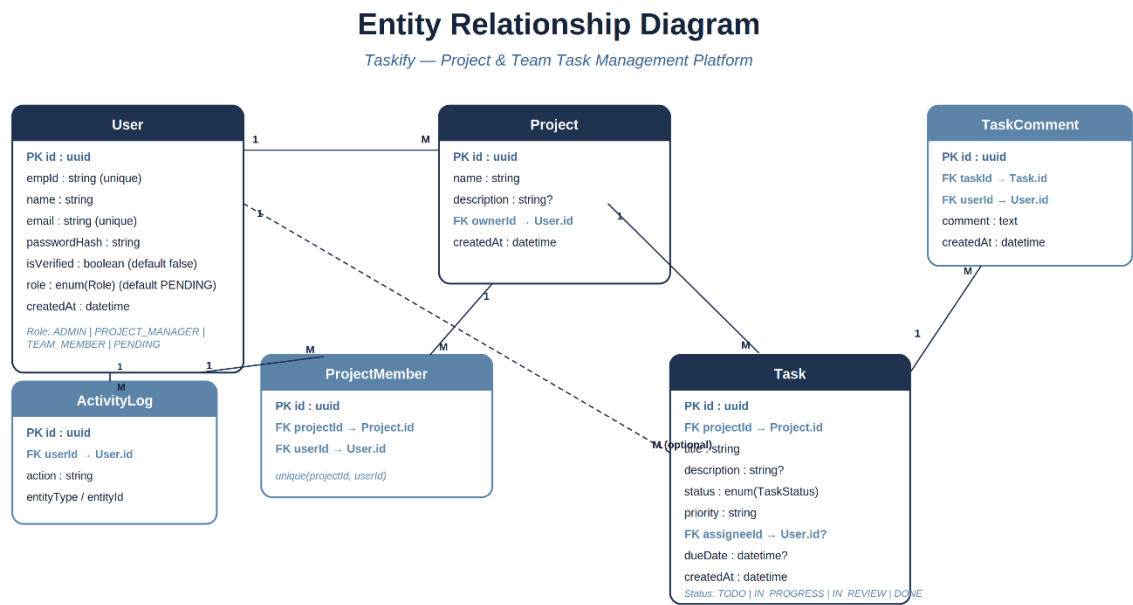


Figure A2: Use case diagram covering Administrator, Project Manager, and Team Member roles

A.3 Entity Relationship Diagram (ERD)



All primary keys are UUIDs. Dashed lines denote optional (nullable) foreign key relationships.

Figure A3: Entity relationship diagram - User, Project, ProjectMember, Task, TaskComment, ActivityLog

Appendix B: AI Tools Disclosure

Claude (Anthropic) was used throughout development as an assistant. The following breakdown describes specifically what it was used for and what was implemented, tested, and validated manually.

Used for

1. Scaffolding the Clean Architecture folder structure and boilerplate (entities, DTOs, repository interfaces) for the Auth, Projects, and Tasks modules.
2. Drafting initial use-case and controller code, including the Edit Task use-case and the Admin approval workflow, which was then reviewed, adjusted, and integration-tested manually.
3. Generating unit test scaffolds for use-cases.
4. Drafting the CustomSelect dropdown component and initial dashboard/login/register UI layouts, later adjusted for the final visual design.
5. Received AI assistance with setting up GitHub Actions CI workflows and troubleshooting CI pipeline issues, including dependency versions, Prisma configuration, and ESLint/TypeScript errors.
6. Drafting this Feature Completion Report, the README, and the system diagrams (architecture, use case, ERD).

Not used for / manually done

1. All business logic decisions, database schema design, and architectural choices were reviewed and validated manually.
2. Debugging of integration issues (authentication flow, RBAC edge cases, Prisma version conflicts) was carried out hands-on.
3. All API testing was performed manually through Swagger UI, not automated or AI-generated.
4. Final verification of the deployed application (Vercel and Render) and the end-to-end user flow was performed manually before submission.